Everything I asked for was built. Two undefined names mean none of it has ever run.

Fifth independent pass. This is the strongest round of the series — the causality evidence is now genuinely measured, and I could not bypass the smoke chain on its merits. But the gate that enforces it crashes before reaching its own checks.

FLOW_BLOCKED

Not for a design flaw. `repo_root` and `hashlib` are undefined in `collect.py`, so `--stage full` raises `NameError` and gate checks 6–10 are unreachable. It fails closed — but the entire hardening is unproven.

FIXED & VERIFIED	NEW CRITICALS	BYPASSES LANDED	TESTS PASSING	NEW WARNINGS
7	1	0 / 5	131	2

EXECUTIVE VERDICT

The first round where I failed to break anything on its merits

Every finding from round 4 was fixed, and several were fixed better than I specified. `triggering_1s_ts_init` is now persisted to the candidate record; the validator's guard is *inverted* so a check that cannot run raises `MISSING_CAUSALITY_METADATA`; and it now reports `causality_rows_examined` and `causality_coverage_pct` alongside the verdict — the "report what you examined" discipline applied exactly as intended. Runs now carry `composite_seal_hash` in their manifest, and both the validator and the gate enforce it. The audit report requires a structured `AUDIT_SUMMARY_V2` block, *and* cross-checks the declared critical count against `### CRITICAL` headings — a defence I did not ask for and which correctly caught my contradiction attempt.

I attacked the smoke chain five ways and landed nothing:

- Pointed the validator at the pre-fix run → `RUN_SEAL_MISMATCH`.
- Forged that run's `run_manifest.json` to carry today's composite → `MISSING_CAUSALITY_METADATA`. **Real defence in depth** — the second layer caught what the first would have missed.

- Prose-only audit report → `MISSING_AUDIT_SUMMARY_V2`.
- Structured summary declaring CLEAR while the prose listed two CRITICALs → `FINDING_COUNT_MISMATCH`.
- Legacy provenance v1 status → `PROVENANCE_V1_RETIRED`.

I also re-measured the current run independently against the raw catalog: **2,002 rows, 0 violations, and 0 in-file `triggering_1s_ts_init != observation_ts` mismatches.** For the first time the framework's own evidence and my independent measurement agree, because the framework is now actually computing it.

> **And then the gate crashes.** `collect.py:90` references `repo_root`; `:93` references `hashlib`. Neither is imported or defined anywhere in the module. Any `--stage full` invocation raises `NameError` at the validator-SHA check — which sits *before* the violation-count, coverage, rows-examined, run-dir and run-manifest checks. Checks 6 through 10 of the new gate have never executed once.

The reason it survived is worth stating precisely, because it is the round-5 form of the pattern this series keeps finding. There *are* two smoke-gate tests. Both supply a deliberately *invalid* acceptance — one missing, one with a wrong composite — and assert the resulting rejection. Both rejections occur before line 90. **The tests prove the gate says no; nothing proves it can say yes.**

ROUND-4 SCORECARD

Re-tested against the live tree

ID	FINDING	EVIDENCE	STATUS
R4-1	Causality check was dead code	Column persisted (`collector:634, :740`); validator raises when absent; adds rows-examined + coverage. Newest run: 73 cols, 0 in-file mismatches, 0 violations by independent catalog measurement	FIXED
R4-2	No run-to-seal binding	`composite_seal_hash` + `execution_manifest_sha256` in `run_manifest.json`; enforced in validator (`RUN_SEAL_MISMATCH` fired against the old run) and in the gate	FIXED
R4-3	Preflight ran 6 of 14 tests; suite uncollectable	Selector globs `test_*.py` → 14 files; broken guardrail test repaired; **full suite: 131 passed** (115 in 61s, plus 16 in the end-to-end runner file taking 16m 48s — see R5-2)	FIXED
R4-4	Feature check weakened to intersection	`declared_metadata` sourced from the spec, plus an explicit surplus-column check	FIXED
R4-5	Causality assert was a tautology	Now a raised exception guarded by a real condition, and made redundant by R4-1's persistence	FIXED

ID	FINDING	EVIDENCE	STATUS
R3-1	Forged smoke acceptance	Gate now checks validator SHA, violation counts, coverage, rows examined, exact equality, run-dir existence and run-manifest binding — **correct by design, unreachable in practice (R5-1)**	BLOCKED BY R
R3-2	Parser optional; v1 accepted	Mechanically fixed: `PROVENANCE_V1_RETIRED`, seal re-parses the report. Fundamentally open: report and status still authored by the same hand; `transcript_sha256: None` accepted	PARTIAL
R3-3	Parser read blocking audits as CLEAR	Requires `AUDIT_SUMMARY_V2` JSON block; raises on missing, duplicate or malformed; cross-checks declared counts against `### CRITICAL` headings	FIXED
F3	Original look-ahead defect	Independently re-verified on run `20260815_152503`: 2,002 / 2,002 exact, 0 violations	HOLDS

BYPASS ATTEMPTS

Five attacks, zero landed

THE SMOKE CHAIN HELD UNDER LAYERED ATTACK

```
$ validate_smoke.py --run-dir runs/20260815_064656_..._day      # the pre-fix run
SMOKE_VALIDATION_FAILED: RUN_SEAL_MISMATCH: run_manifest composite_seal_hash (None)
  != current verified seal (25bb58f9ee62acfb...)

$ # forge that run's manifest to carry today's composite (runs/ is unsealed), retry
forged run_manifest.json on the pre-fix run
SMOKE_VALIDATION_FAILED: MISSING_CAUSALITY_METADATA: candidate records missing
  'triggering_1s_ts_init' column          <- second layer caught it
```

THE AUDIT PATH HELD TOO

```
prose-only report              -> MISSING_AUDIT_SUMMARY_V2
summary CLEAR + 2 prose CRITICALs -> FINDING_COUNT_MISMATCH: Report contains 2 CRITICAL
                                     headings, but summary claims critical=0
legacy provenance v1 status    -> PROVENANCE_V1_RETIRED
```

BUT THE GATE NEVER GETS THAT FAR

```
$ # hand-written acceptance satisfying every documented gate check
BLOCKED: NameError | name 'repo_root' is not defined

$ python -c "symtable analysis of collect.py"
globals referenced but not module-level: ['repo_root', 'hashlib']
  repo_root: module-level defined = False
  hashlib:   module-level defined = False
```

INDEPENDENT RE-MEASUREMENT — FRAMEWORK EVIDENCE NOW MATCHES REALITY

```
newest run: 20260815_152503_..._day
  rows: 2002  cols: 73  has triggering_1s_ts_init: True
```

```
INDEPENDENT catalog check -> violations: 0 / 2002
in-file triggering != observation:      0
```

FINDINGS

One critical, one carried-forward governance item

CRITICAL R5-1

The hardened smoke gate raises NameError before reaching any of its new checks

backtests/nt_runtime/modes/collect.py:90 (`repo_root`), :93 (`hashlib`) · tests: scripts/tests/test_round2_invariants.py:137, :156

MECHANISM	Check 6 of the gate opens with `val_script_path = repo_root / "scripts" / "validate_smoke.py"` and then `hashlib.sha256(...)`. Neither name is imported at module level nor bound in `run_collect_mode` — symtable analysis confirms both resolve as undefined globals. `json` is imported inline inside the function and is fine. Execution therefore terminates at line 90 with `NameError`, leaving checks 6–10 — validator SHA, `future_source_violations_count`, `causality_coverage_pct`, `causality_rows_examined`, `exact_timestamp_equality_verified`, run-dir existence and run-manifest seal binding — permanently unreachable.
FAILURE	`--stage full` is non-functional. It fails closed, so nothing unsafe executes; the harm is that the entire R3-1 remediation is unproven, and an operator meeting `NameError: name 'repo_root' is not defined` gets no actionable signal — it reads as a broken tool rather than a policy decision, which invites working around it.
WHY MISSED	Both smoke-gate tests construct a *deliberately invalid* acceptance — `test_full_stage_rejects_missing_smoke_acceptance` removes the file, `test_full_stage_rejects_stale_smoke_acceptance` sets `sealed_composite_sha256: "stale_or_wrong_composite_hash"` — and assert the resulting rejection. Both rejections fire at lines 44 and ~70, before the undefined name. No test supplies a valid acceptance, so the accept path has never been executed by anything.
FIX	Add `import hashlib` at module level and define `repo_root = Path(__file__).resolve().parents[3]` (or take it from `study_data`) near the top of the gate. Two lines.
REGRESSION TEST	`test_full_stage_accepts_valid_smoke_acceptance` — build a genuine acceptance from the current seal and a real run manifest, monkeypatch `build_engine` to a sentinel, and assert the sentinel is reached. This is the missing half of the existing pair: every gate needs one test that it admits the valid case, not only that it rejects invalid ones. Add the same for `test_seal_covers_full_import_closure`'s neighbours where only rejection is covered.

WARNING R5-2

The test-selection fix pulled a 10-minute market replay into preflight, which has no subprocess timeout

`scripts/select_required_tests.py:58` · `scripts/research_preflight.py:159–163` · `scripts/tests/test_nt_runner_collect.py:349`

MECHANISM

Replacing the hardcoded six-file list with `test_dir.glob("test_*.py")` correctly fixed R4-3 — but it also swept in `test_nt_runner_collect.py`, which contains `test_end_to_end_1day_collect_run_nonzero_candidates`: a real NautilusTrader replay of a full day. Preflight then runs the whole selection via `subprocess.run(pytest_cmd, capture_output=True, text=True, cwd=...)` with **no `timeout=` argument**.

FAILURE

Measured, and everything passes — the problem is the cost, not the result:

```
scripts/tests/test_nt_runner_collect.py   16 passed in 1008.03s (0:16:48)
all other test files                      115 passed in 60.89s
                                          ──────────────────────────
full suite                                131 passed, ~17m 49s
```

Preflight's `CAUSAL_INVARIANTS` stage was 8.65s in round 4 under the old six-file selector. It now runs all 14 files, so **every preflight invocation carries a ~17-minute market replay**. Because the subprocess has no timeout, a replay that hangs hangs preflight indefinitely with no diagnostic. `CLAUDE.md` specifies preflight as the *fast* deterministic gate — "fast causal canaries for zero LLM tokens".

WHY IT MATTERS

The predictable response to a gate that takes ten minutes is `--skip-tests`, permanently. That silently disables `CAUSAL_INVARIANTS` and re-creates the exact R4-3 condition — regression tests not running under any gate — by a different route. A remediation that makes the gate impractical tends to undo itself.

FIX

Mark the end-to-end tests `@pytest.mark.slow` and have preflight deselect them by default (`-m "not slow"`) while the pre-execution gate runs the full set explicitly. Independently, pass `timeout=` to `subprocess.run` and treat expiry as a `BLOCKED` verdict with a distinct failure id — a gate that can hang forever is not a deterministic gate.

REGRESSION TEST

`test_preflight_fast_stage_completes_under_budget` — assert the default preflight invocation returns within a stated wall-clock budget.

WARNING F2

Audit independence remains an assertion, not a property — five rounds on

`scripts/run_preexec_audits.py:104-150` · `scripts/preexec_audit_seal.py:84-130` · `.claude/agents/lookahead-auditor.md:3`

MECHANISM

The parsing layer is now genuinely good: structured summary required, counts cross-checked against headings, v1 retired, seal re-derives from the report rather than trusting the JSON. But the report is written by whoever runs the flow. I hand-authored a `pass_13.md` containing "I audited nothing at all" plus a well-formed `AUDIT_SUMMARY_V2` block declaring CLEAR, hand-wrote both v2 status files to match, and obtained `seal_status: LOCKED`. `transcript_sha256: None` is accepted. Separately, `lookahead-auditor` still holds `Bash` and `Write`, and `settings.local.json` still has no `hooks` key, so the `results-triager` guard on disk never runs.

ASSESSMENT

This is not a bug and further parser work will not close it. Independence cannot be established cryptographically by the party being audited. The remaining decision is a policy one, and it has been open since round 2.

FIX

Pick one. **(a)** Require `transcript_sha256` to be non-null and bound to an auditor invocation record the orchestrator does not author, and have the seal reject without it. **(b)** Accept self-attestation and rename the artifacts to `self_attested_causal_review.json`, dropping "independent" from `CLAUDE.md`, `AGENTS.md` and the seal's field names. Option (b) costs nothing and makes the guarantee honest; option (a) is the real control. What should not persist is machinery that reads as independent verification while being self-certification.

REGRESSION TEST

For (a): `test_seal_requires_auditor_transcript`. For (b): a docs assertion that no artifact name or field claims independence.

VERIFIED SOUND —

What now genuinely works

measured this round

CAUSALITY	The collector is correct and the framework can now *prove* it. Independent catalog measurement and the in-file `triggering_1s_ts_init` column agree: 2,002 / 2,002 exact, 0 violations.
DEFENCE IN DEPTH	Demonstrated, not assumed: forging the run-manifest binding got past layer one and was stopped by layer two. That is the first time in this series a second layer has caught something the first missed.
SEAL & CLOSURE	53-file AST closure; `PREEXEC_AUDIT_SEAL_VALID`; preflight `CLEAR`; lint at 100% coverage; the suite is collectable again and **all 131 tests pass** — including the 16 end-to-end runner tests. Nothing is failing anywhere; R5-2 is purely about how long the fast gate now takes.

WHERE THIS LEAVES THE FRAMEWORK

Two lines and one decision

Setting R5-1 aside as the two-line defect it is, this round marks a real inflection: **the causal and evidentiary layers are now sound, and I could not break either on its merits.** The remaining item is F2, and it is a policy choice rather than an engineering gap.

1. **R5-1** — add `import hashlib`, define `repo_root`. Then write the accept-path test, because that omission is what hid it.
2. **R5-2** — mark the end-to-end tests `slow`, deselect them from the fast preflight stage, and give the pytest subprocess a `timeout=`. Otherwise the practical response to a ten-minute gate is to disable it.
3. **F2** — decide between binding a real auditor transcript and renaming the artifacts to reflect self-attestation.

One recommendation on testing convention, since it is now the load-bearing weakness. Across five rounds, the controls that failed were nearly always ones whose tests asserted only refusal: the lint that scanned one file and reported clean, the closure test that checked seven filenames, the `coverage_pct` literal, the acceptance reporting an uninitialised zero, and now a gate that has never admitted a valid input. **Pair every rejection test with an acceptance test.** A control tested only on its failure path can be

entirely broken and still look green — which is precisely how `NameError` came to sit in the middle of the most carefully specified gate in the system.

Red team round 5, 2026-08-15 · study `Gemini_clean_maturity_flip_rolling_5m_productivity` · 53 sealed files · 115 tests passing · run `20260815_152503` independently verified at 2,002 / 2,002 exact. All bypasses executed against the live working tree and reverted; `PREEXEC_AUDIT_SEAL_VALID`, `PREFLIGHT  CLEAR` and a 27-file modified set (unchanged from session start) reconfirmed after restore. The genuine `smoke_acceptance.json` and the pre-fix run's `run_manifest.json` were backed up before testing and restored intact.